

# Processamento de dados com Apache Spark

Integração PostgreSQL, MongoDB e Delta Lake: UA 3 e 4

**Curso:** Pós-Graduação em Data Science e Analytics

**Disciplina:** Banco de Dados e Big Data para Data Science

**Equipe:** Diogo Galvão Carvalho; Felipe Artur Macedo Lima; Luan Cavalcante Dias Rodrigues

**Data:** Setembro de 2026

**Repositório:** [https://github.com/FelipeArtur/bigdata\\_datascience](https://github.com/FelipeArtur/bigdata_datascience)

## 1. Introdução

Este relatório dá continuidade à migração descrita nas UAs 1 e 2. O mesmo cenário de compras de itens em partidas de League of Legends é usado para demonstrar ingestão de documentos MongoDB, enriquecimento com CSV demográfico, transformações em Apache Spark, análise de desempenho e persistência em Delta Lake.

O trabalho acompanha os dados desde a origem relacional até o armazenamento final, passando pelo modelo documental e pelo processamento analítico. A avaliação reúne métricas da amostra, comparação dos dados entre etapas e tempos medidos em um experimento de cache e broadcast join.

Spark distribui o processamento quando o volume ou a complexidade das operações exigem distribuição de trabalho. Seu driver coordena a aplicação, o plano lógico descreve as operações, o otimizador Catalyst seleciona estratégias e tarefas são executadas sobre partições. Persistir um DataFrame pode evitar recomputar etapas compartilhadas; broadcast join pode evitar o embaralhamento da tabela maior ao replicar a dimensão menor.

A integração de fontes é outra vantagem: neste projeto, documentos aninhados e dados tabulares são convertidos em DataFrames e tratados com a mesma API. Delta Lake acrescenta um log transacional aos arquivos Parquet, permitindo gravações consistentes e verificação posterior da saída. O efeito dessas escolhas no desempenho depende da carga de trabalho.

A base de 1.043 compras é adequada à demonstração funcional, mas não caracteriza um teste de grande escala. A execução local [2] usa recursos de uma única máquina; não permite medir comunicação entre servidores ou demonstrar elasticidade de um cluster. Os resultados de desempenho são restritos a esse ambiente.

**Adaptação ao roteiro:** o enunciado solicita Atlas e Databricks Community Edition. A execução foi realizada integralmente em Docker local, por escolha do projeto. Community Edition foi substituída pela Free Edition, cujo ambiente serverless possui limitações distintas. Não houve execução em nenhum desses serviços. Essa adaptação deve ser aceita pelo docente; funcionamento local não comprova cumprimento literal do requisito de nuvem.

## 2. Cenário e modelo de dados

### 2.1 Origem relacional

A base possui sete tabelas e mantém o conjunto de dados da Entrega 1. Jogadores realizam compras em partidas; cada item pertence a uma ou mais categorias. Chaves primárias e estrangeiras representam os vínculos no PostgreSQL.

| Tabela         | Chave / vínculos                           | Registros |
|----------------|--------------------------------------------|-----------|
| elo            | id_elo                                     | 10        |
| categoria      | id_categoria                               | 32        |
| item           | item_id                                    | 254       |
| item_categoria | item_id, id_categoria                      | 834       |
| jogador        | id_jogador; id_elo                         | 25        |
| partida        | id_partida                                 | 30        |
| compra         | id_compra; id_jogador, id_item, id_partida | 1.043     |

Além das chaves da tabela, os atributos são: elo.elo; categoria.nome\_categoria; item.nome, preco\_unitario e descricao; jogador.nick e regioao; partida.data\_partida, hora\_partida, duracao\_minutos e resultado; compra.data\_compra, minuto\_compra, quantidade, preco\_unitario e total\_compra. A associação item\_categoria contém suas duas chaves.

```
elo (1) — (N) jogador (1) — (N) compra (N) — (1) partida
                        | (N)
                        | (1)
                        item
                        | (1)
                        | (N)
                    item_categoria
                        | (N)
                        | (1)
                        categoria
```

compra também contém data, minuto, quantidade, preço unitário e total. O preço da transação é independente de mudanças posteriores no catálogo. O total é um valor redundante protegido por CHECK (total\_compra = quantidade \* preco\_unitario).

### 2.2 Origem e significado dos valores

O catálogo bruto registra versão 16.19.1 do Data Dragon, coletada em 25/09/2026. Partidas, compras, perfis demográficos e tiers de assinatura são sintéticos, gerados ou definidos para este exercício. A seed 42 permite reproduzir os CSVs. Seis itens de preço zero recebem valor simbólico de 150 ouro. Essa transformação pertence à simulação e não deve ser confundida com preço oficial.

O resultado global da partida é apenas ilustrativo: não existe entidade de equipe no modelo. Não são feitas análises de vitória individual. Ouro é movimentação de recurso virtual, sem conversão em receita monetária. Os dados não autorizam conclusões sobre pessoas reais, preferências competitivas ou efeito causal de assinaturas.

## 2.3 Modelo documental retomado da Entrega 1

Compras e elo textual são embutidos em jogadores, porque o padrão de acesso selecionado é consultar perfil e histórico em conjunto. Catálogo e partidas são documentos independentes, referenciados pelos IDs das compras. Categorias são embutidas nos itens. Um documento dominios conserva todos os elos e categorias, inclusive valores não utilizados.

Cada compra preserva o preço da transação e recebe o nome do catálogo no instante da migração. A carga substitui cada coleção após validar seu conteúdo temporário. As trocas são independentes e não formam uma única transação entre coleções.

Exemplos abreviados da transformação usada nesta execução:

### jogadores (recorte):

```
{"_id": 1, "nick": "Faker", "elo": "Desafiante", "total_transacoes": 37, "compras": [{"id_compra": 72, "id_item": 3508, "id_partida": 3, "data_compra": "2025-08-21", "minuto_compra": 18, "quantidade": 1, "preco_unitario": 3050, "item": "Colhedor de Essência", "total_ouro": 3050}]}
```

### itens (recorte):

```
{"_id": 3508, "nome": "Colhedor de Essência", "preco_unitario": 3050, "categorias": [{"id_categoria": 1, "nome_categoria": "AbilityHaste"}]}
```

### partidas (recorte):

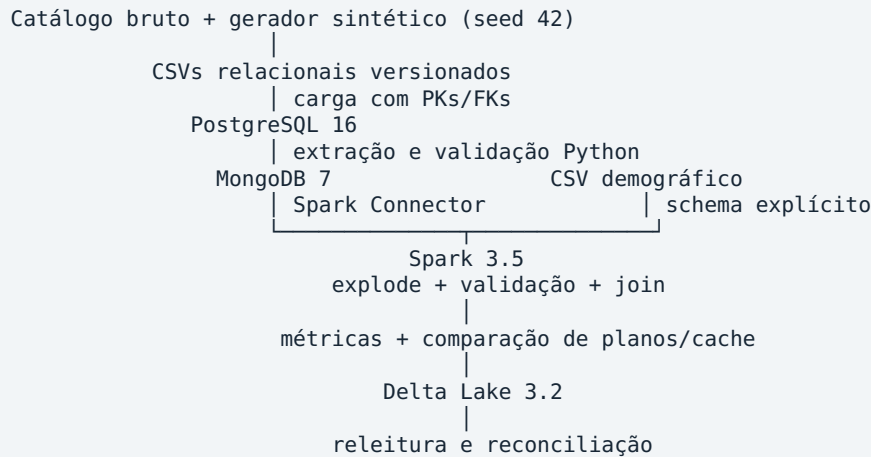
```
{"_id": 3, "id_partida": 3, "data_partida": "2025-08-21", "hora_partida": "20:47:00", "duracao_minutos": 35, "resultado": "Vitoria"}
```

### dominios (recorte):

```
{"_id": "dominios", "elos": [{"id_elo": 1, "elo": "Ferro"}]}
```

Para uma aplicação com histórico extenso, o array de compras precisaria ser limitado ou separado. O limite de 16 MiB por documento e a concentração de escritas são restrições relevantes. Sharding não foi configurado nem medido. Referências entre coleções são verificadas pela aplicação, não por FKs do MongoDB.

### 3. Arquitetura e ingestão



São usados três containers: PostgreSQL, MongoDB e o ambiente Python/Java com Jupyter. Os bancos ficam na rede interna do Compose. Jupyter é exposto apenas em 127.0.0.1 e utiliza token. A imagem fixa as versões das dependências Python diretas e resolve os JARs compatíveis durante a construção. A primeira construção requer acesso à internet.

A execução local usa duas threads de processamento na mesma máquina. O experimento configura quatro partições de shuffle; outras cargas podem exigir um ajuste diferente. Os DataFrames mantêm avaliação preguiçosa: ações como collect e escrita materializam o trabalho.

A ingestão lê o MongoDB pelo conector. Se a leitura falhar, o pipeline interrompe a execução, sem substituir a origem por JSON:

```
raw = (spark.read.format("mongodb")
      .option("connection.uri", os.environ["MONGODB_URI"])
      .option("database", os.environ["MONGODB_DATABASE"])
      .option("collection", "jogadores")
      .load())

demo = (spark.read.schema(schema_demografico)
      .option("header", True).option("mode", "FAILFAST")
      .csv("data/processed/jogadores_demografia.csv"))
```

O código completo fixa uma única partição de leitura MongoDB para esta amostra pequena. O schema do CSV declara IDs e idade inteiros; demais campos são texto. O modo de leitura é estrito, evitando inferência baseada apenas nas primeiras linhas.

## 4. Transformações e métricas

O array `compras` é desaninhado com `explode`. IDs, valor e data são selecionados para a tabela analítica. Antes do join, o pipeline rejeita IDs demográficos nulos ou duplicados, compras sem demografia, IDs de compra repetidos e datas inválidas. Isso evita que um inner join esconda perdas de linhas ou multiplique transações.

```
fact = raw.select("id_jogador", "nick", "regiao", "elo",
                  F.explode("compras").alias("c"))
# A seleção seguinte expõe os campos da compra e converte a data.
enriched = purchases.join(
    F.broadcast(demo.drop("nick", "regiao")), "id_jogador", "inner")

by_region = enriched.groupBy("regiao", "elo").agg(
    F.sum("total_ouro").alias("total_ouro"),
    F.sum("quantidade").alias("unidades"),
    F.count("*").alias("transacoes"),
    F.round(F.avg("total_ouro"), 2).alias("ticket_medio_ouro"))
```

Quatro análises são implementadas: movimentação por região/elo; produtos por quantidade e por ouro; transações por dia da semana/plataforma; e assinaturas em período delimitado. O filtro temporal usa o intervalo inclusivo de 20/08/2025 a 20/09/2025. Os rankings por volume e por valor são separados; desempates são resolvidos pelo ID do item para garantir estabilidade.

O pipeline processou 1043 compras, com 1204 unidades e 2228821 ouro. Sábados e domingos concentraram 688491 ouro (30.89%).

| Região | Elo         | Ouro   | Transações | Ticket  |
|--------|-------------|--------|------------|---------|
| KR     | Desafiante  | 351612 | 170        | 2068.31 |
| BR     | Mestre      | 335025 | 144        | 2326.56 |
| BR     | Diamante    | 330861 | 156        | 2120.9  |
| KR     | Grão-Mestre | 318825 | 136        | 2344.3  |
| KR     | Mestre      | 227783 | 107        | 2128.81 |
| BR     | Esmeralda   | 162374 | 82         | 1980.17 |
| BR     | Platina     | 160800 | 89         | 1806.74 |
| BR     | Ouro        | 116283 | 59         | 1970.9  |
| EUW    | Mestre      | 90350  | 43         | 2101.16 |
| BR     | Grão-Mestre | 78000  | 31         | 2516.13 |
| EUW    | Grão-Mestre | 56908  | 26         | 2188.77 |

| Item (top 5 por unidades) | Unidades | Ouro  |
|---------------------------|----------|-------|
| Lacre Sombrio             | 14       | 4900  |
| Broto de Esmagamusgo      | 11       | 4950  |
| Elixir de Ferro           | 11       | 5500  |
| Concretizador             | 11       | 30800 |
| Morellonomicon            | 11       | 31350 |

Esses valores descrevem a amostra gerada. Diferenças entre elos não demonstram que habilidade cause maior gasto. A predominância de um tier também pode refletir a composição manual dos perfis, e não o efeito de uma assinatura.

## 4.1 Temporalidade, segmentação e interpretação

```
weekday = enriched.withColumn("dia_semana",
    F.dayofweek("data_compra")).groupBy("dia_semana", "plataforma").agg(
    F.sum("total_ouro").alias("total_ouro"),
    F.count("*").alias("transacoes"))

subscription = enriched.filter(
    F.col("data_compra").between("2025-08-20", "2025-09-20"))
    .groupBy("tierassinatura").agg(
    F.sum("total_ouro").alias("total_ouro"),
    F.count("*").alias("transacoes"))
```

| Tier     | Ouro no período | Transações | Ticket  |
|----------|-----------------|------------|---------|
| VIP      | 1185024         | 518        | 2287.69 |
| Gratuito | 430520          | 224        | 1921.96 |
| Pro      | 275300          | 127        | 2167.72 |

| Dia     | Ouro   | Transações |
|---------|--------|------------|
| Domingo | 389433 | 173        |
| Segunda | 328375 | 151        |
| Terça   | 344140 | 168        |
| Quarta  | 208500 | 109        |
| Quinta  | 368241 | 163        |
| Sexta   | 291074 | 141        |
| Sábado  | 299058 | 138        |

VIP e Pro somam **1460324 de 1890844 ouro (77.23%)** no período filtrado.

| Item (top 5 por ouro) | Ouro  | Unidades |
|-----------------------|-------|----------|
| Morellonomicon        | 31350 | 11       |
| Concretizador         | 30800 | 11       |
| Força da Natureza     | 28000 | 10       |
| Colhedor de Essência  | 27450 | 9        |
| Cutelo Negro          | 27000 | 9        |

A participação de fins de semana usa como denominador todo o ouro da amostra. A participação de VIP e Pro usa apenas o período filtrado. As duas proporções, portanto, usam bases de cálculo diferentes.

Ouro por transação é a média de `total_ouro`, não a média de preço unitário. Como algumas compras têm quantidade dois, as métricas não são intercambiáveis. Quantidade vendida e frequência de compras também são diferentes: uma transação pode envolver várias unidades.

A análise temporal agrega datas simuladas de poucas semanas. Não permite concluir sazonalidade recorrente ao longo do ano. Atribuir um efeito a sessões prolongadas ou ao tipo de computador exigiria variáveis adicionais e um desenho de análise que este trabalho não possui.

## 5. Otimização: metodologia e resultados

O benchmark executa a mesma agregação por região/elo e verifica resultados idênticos em todas as variantes. Há cinco repetições medidas após aquecimento. As durações brutas, as medianas, as versões do ambiente e os planos físicos são persistidos em `reports/evidence`.

Na comparação de joins, o experimento desativa AQE e broadcast automático para comparar um `SortMergeJoin` observado com um `BroadcastHashJoin` explícito. As execuções são alternadas para reduzir viés de ordem. Cada consulta é reconstruída para não reutilizar resultados de shuffle da mesma execução física. As configurações anteriores são restauradas ao final. Fora desse experimento, Spark pode escolher broadcast automaticamente; retirar o hint não garante um plano sort-merge.

Para avaliar o cache, a mesma consulta sobre o join broadcast é medida sem cache e com cache materializado. O tempo para materializar o `DataFrame` é registrado separadamente e não incluído no tempo de reutilização. A comparação é sequencial, com aquecimento; ainda pode sofrer efeitos de JVM, I/O e carga da máquina. Não se mede somente `count()` nem se extrapola o resultado para todas as análises.

| Variante   | Mediana (s) |
|------------|-------------|
| sort_merge | 0.287043    |
| broadcast  | 0.268514    |
| uncached   | 0.231440    |
| cached     | 0.154836    |

| Repetição | Sort-merge | Broadcast | Sem cache | Com cache |
|-----------|------------|-----------|-----------|-----------|
| 1         | 0.328877   | 0.271510  | 0.262948  | 0.141414  |
| 2         | 0.287043   | 0.258913  | 0.231440  | 0.154836  |
| 3         | 0.272794   | 0.268514  | 0.252592  | 0.164746  |
| 4         | 0.338211   | 0.280233  | 0.207613  | 0.197626  |
| 5         | 0.272439   | 0.236607  | 0.230016  | 0.153068  |

Materialização do cache: **0.338661 s**. Razão sem/com cache: **1.49x**; razão sort-merge/broadcast: **1.07x**. As razões não incluem o custo inicial de materializar o cache.

A mediana com cache foi menor à da consulta sem cache nesta execução.

A mediana com broadcast foi menor à da consulta com sort-merge nesta execução.

Os tempos se referem a esta consulta e a este ambiente; não permitem concluir que o ganho se repita em grande escala. O cache precisa ser reutilizado para compensar sua materialização.

## 5.1 Planos físicos e limites da otimização

Os arquivos `plan_sort_merge.txt`, `plan_broadcast.txt` e `plan_cached.txt` registram os planos efetivamente obtidos. Os operadores relevantes são:

```
SortMergeJoin: junção com ordenação e redistribuição pelas chaves.  
BroadcastHashJoin: dimensão replicada; evita shuffle da tabela maior no join.  
InMemoryTableScan: consulta reutiliza o DataFrame materializado.
```

Broadcast não elimina todo o tráfego: a dimensão ainda precisa ser distribuída. Agregações posteriores podem introduzir novos exchanges. Cache também não é gratuito: consome memória e tem um custo inicial, só compensado quando houver reutilização suficiente. O código libera os dados persistidos após o experimento.

A vantagem medida é uma razão entre medianas. Razão acima de 1 indica menor tempo da variante otimizada; abaixo de 1 indica que ela foi mais lenta nesta execução. O relatório registra a razão obtida mesmo quando a variante otimizada é mais lenta. A amostra é pequena e o overhead do motor pode dominar o custo útil do processamento.

## 6. Persistência e conservação dos dados

A saída é gravada em Delta Lake e relida. Nesta amostra, não foi aplicado particionamento por região: poucos registros divididos em diretórios adicionais gerariam arquivos pequenos sem benefício demonstrado.

```
enriched.write.format("delta").mode("overwrite").save(path)  
loaded = spark.read.format("delta").load(path)  
assert enriched.exceptAll(loaded).limit(1).count() == 0  
assert loaded.exceptAll(enriched).limit(1).count() == 0
```

Antes da escrita, todos os campos transacionais das compras extraídas pelo Spark são comparados com PostgreSQL. Depois dela, `exceptAll` nas duas direções verifica igualdade com multiplicidade, não apenas contagens. O `overwrite` aplica-se à tabela analítica de demonstração; não é uma migração incremental nem uma política de retenção histórica.

| Evidência              | Resultado                                                |
|------------------------|----------------------------------------------------------|
| Data da execução (UTC) | 2026-09-29T00:05:37.792514+00:00                         |
| Origem Spark           | mongodb_connector                                        |
| PostgreSQL × Spark     | Todos os campos de compra iguais                         |
| Spark × Delta          | Diferenças vazias nas duas direções                      |
| Linhas relidas         | 1043                                                     |
| Versões                | Python 3.12.14; Spark 3.5.3; Delta 3.2.0; Java 17.0.20.1 |
| Execução               | local[2]                                                 |

## 7. Conclusão e avaliação crítica

A leitura do MongoDB, a junção com o CSV demográfico e a gravação em Delta foram executadas e verificadas. A comparação dos campos confirmou a conservação das compras entre PostgreSQL, Spark e Delta. Os notebooks e os arquivos de evidência registram os resultados e permitem reproduzir o experimento.

Cada escolha tem um alcance específico. Embedding reúne dados consultados em conjunto, sem comprovar menor latência. Broadcast evita o shuffle da tabela maior no join, embora ainda transmita a dimensão. O cache exige materialização e memória para reduzir o custo das consultas seguintes. A escrita transacional do Delta abrange a tabela, não o fluxo inteiro.

As técnicas foram exercitadas em uma máquina e numa amostra sintética. O experimento não demonstra ganhos em múltiplos servidores, alta concorrência, tolerância a falhas de infraestrutura ou comportamento econômico real. O fluxo de migração entre coleções MongoDB também pressupõe uma janela sem leitores concorrentes.

Uma próxima avaliação poderia manter o contrato de dados e aumentar o volume, medir memória e shuffle, testar particionamento e verificar a recuperação após falhas. Essas condições ainda precisam ser testadas.

O fluxo documentado foi executado por completo no ambiente local. A substituição de Atlas/Databricks deve ser validada pelo docente, pois os roteiros os citam explicitamente. Cada integrante deve realizar a entrega individual do PDF correspondente no AVA.

## Referências

APACHE SOFTWARE FOUNDATION. **Spark SQL performance tuning**. Apache Spark 3.5.3. Disponível em: <https://spark.apache.org/docs/3.5.3/sql-performance-tuning.html>. Acesso em: 28 set. 2026.

DELTA LAKE. **Quick start**. Disponível em: <https://docs.delta.io/quick-start/>. Acesso em: 28 set. 2026.

DELTA LAKE. **Releases: compatibility with Apache Spark**. Disponível em: <https://docs.delta.io/releases/>. Acesso em: 28 set. 2026.

MONGODB, INC. **Getting started with the Spark Connector**. Disponível em: <https://www.mongodb.com/docs/spark-connector/v10.x/getting-started/>. Acesso em: 28 set. 2026.

DATABRICKS. **Sign up for Databricks Free Edition**. Disponível em: <https://docs.databricks.com/aws/en/getting-started/free-edition>. Acesso em: 28 set. 2026.

DATABRICKS. **Serverless compute limitations**. Disponível em: <https://docs.databricks.com/aws/en/compute/serverless/limitations>. Acesso em: 28 set. 2026.